

Course: Data Structures (CSE CS203A)

Assignment V: Tree

Student ID : 1131427 Student Name : 田秉立

Section 1. Definitions of Tree Variants

1. General Tree

Definition:

一般樹是一種階層式 (hierarchical) 的資料結構，由一個根節點 (root) 開始，每個節點可以有 **任意數量的子節點 (children)**。

重點特性

- 每個節點可有 0 個或多個子節點
- 沒有左右順序限制
- 常用來表示組織架構、檔案系統

2. Binary Tree

Definition:

二元樹是一種樹結構，其中**每個節點最多只能有兩個子節點**，分別稱為**左子樹 (left child)** 與 **右子樹 (right child)**。

重點特性

- 每個節點：最多 2 個子節點
- 左右子樹有順序
- 不要求填滿

3. Complete Binary Tree

Definition:

Complete Binary Tree 是指除了最後一層以外，每一層都**完全填滿**，而最後一層的節點從**左到右依序填入**。

重點特性

- 結構接近 Full Binary Tree
- 節點排列有順序 (左到右)
- Heap 一定是 Complete Binary Tree

4. Binary Search Tree(BST)

Definition:

BST 對每個節點都滿足：

- 左子樹所有節點的值 **小於** 該節點
- 右子樹所有節點的值 **大於** 該節點

重點特性

- 搜尋、插入、刪除平均時間： $O(\log n)$
- **可能退化成 LinkList (最壞 $O(n)$)**

5. AVL Tree

Definition:

AVL Tree 是一種**優化的 BST**，對每個節點都滿足左右子樹高度差**不超過 1**，避免不平衡或退化

重點特性

- 屬於 BST
- 插入或刪除後，會進行旋轉維持平衡
- 保證高度為 $O(\log n)$
- 搜尋速度快，但旋轉成本較高

6. Red-Black Tree

Definition:

紅黑樹是一種優化的 BST，每個節點都有顏色（紅或黑），並滿足特定的顏色規則來維持平衡，避免不平衡或退化

五大性質

1. 每個節點不是紅色就是黑色
2. 根節點是黑色
3. 所有葉節點（NULL）是黑色
4. 紅色節點不能有紅色子節點
5. 從任一節點到其所有葉節點的黑色節點數相同

重點特性

- 平衡條件較 AVL 寬鬆
- 插入刪除旋轉次數較少

- 實務常用 (如 map, set)

7. Max Heap

Definition:

Max-Heap 是一種 Complete Binary Tree，且滿足每個節點的值都 **大於或等於** 其子節點

重點特性

- 根節點為**最大值**
- 常用於 Priority Queue (優先佇列)

8. Min Heap

Definition:

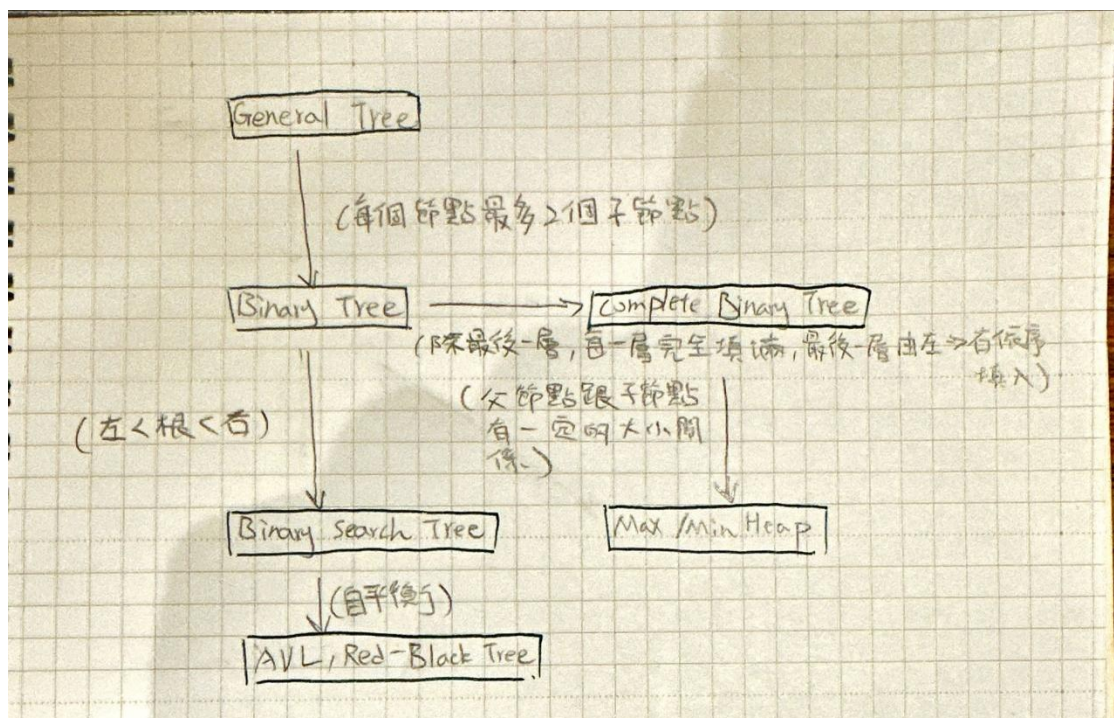
Min-Heap 是一種 Complete Binary Tree，且滿足每個節點的值都 **小於或等於** 其子節點

重點特性

- 根節點為**最小值**
- 插入、刪除最小值： $O(\log n)$
- 常用於急救順序排程問題

Section 2. Tree Family Hierarchy and Transformations

2.1 Tree Family Diagram



2.2 Explananation of Transformations

From	To	New property / constraint added
General Tree	Binary Tree	每個節點最多 2 個子節點
Binary Tree	Complete Binary Tree	除了最後一層外，每一層都要填滿，最後一層必須 從左到右填
Binary Tree	BST	左子樹所有節點 < 根節點 右子樹所有節點 > 根節點
BST	AVL Tree	任一節點左右子樹高度差 ≤ 1
BST	Red-Black Tree	用顏色（紅 / 黑）限制，使其盡量平衡
Binary Tree	Max Heap	父節點 $\geq$ 子節點
Binary Tree	Min Heap	父節點 $\leq$ 子節點

Section 3. Tree Construction Using Given Integers

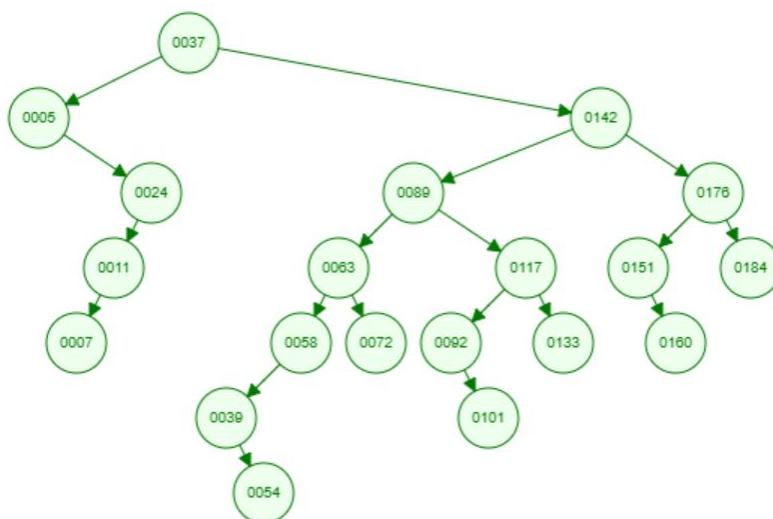
3.1 Binary Tree

Tool name / URL: <https://tree-visualizer.netlify.app/>

Construction / insertion description:

在 binary tree（二元樹）中，如果沒有額外限制（像 BST 的大小比較規則），插入規則通常是從上到下、從左到右填滿空位，也就是遇到第一個空的左子節點就插左，左滿才插右，右滿才往下一層；如果依照給定順序插入 20 個整數，就會依序把每個整數放到當前節點的左子樹或右子樹，確保上一層的節點先填滿左右子樹，再往下一層繼續，這樣左右子樹的安排完全依照插入順序和「層序填滿」的規則，而不是比較大小，不過因為 BST 也是一種 binary tree 故下圖以 BST 為例。

Screenshot of Binary Tree (paste below):



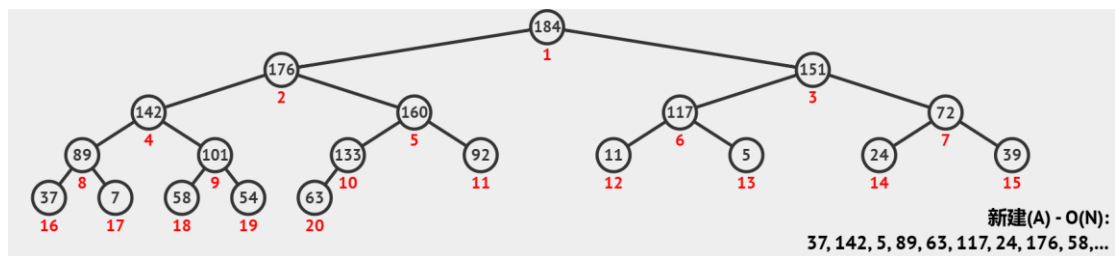
3.2 Complete Binary Tree

Tool name / URL: <https://visualgo.net/zh/heap>

Construction / insertion description:

要用給定的 20 個整數建立 complete binary tree，就是從第一個數字開始當根節點，然後依序往左子節點、右子節點填，填滿一層才往下一層，最底層也是從左到右依序放，這樣整棵樹就會除了最底層可能沒滿以外，每一層都滿，而且底層的節點都集中在左邊，符合 complete binary tree 的規則。

Screenshot of Complete Binary Tree (paste below):



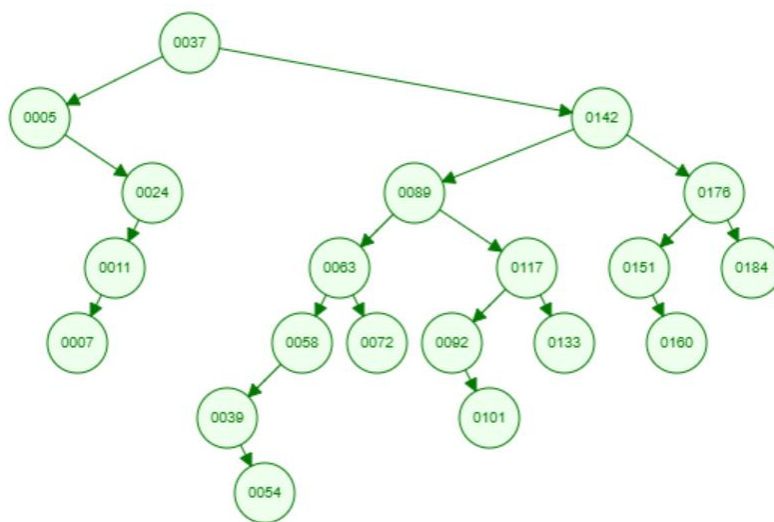
3.3 Binary Search Tree (BST)

Tool name / URL: <https://www.cs.usfca.edu/~galles/visualization/BST.html>

Insertion rule (e.g., “insert in given order using BST rules”):

在二元搜尋樹 (BST) 中插入新整數時，會從根節點開始，將欲插入的值與目前節點比較，若較小則往左子樹，較大則往右子樹，並在該方向的子節點繼續重複比較，直到遇到空的位置 (null) 為止，最後將新值插入該位置成為葉節點；因此，BST 中左右子樹的決定完全取決於「與當前節點的大小比較」，而插入順序也會直接影響整棵樹的形狀與高度，如果順序插入 BST 會退化成 linklist

Screenshot of BST (paste below):



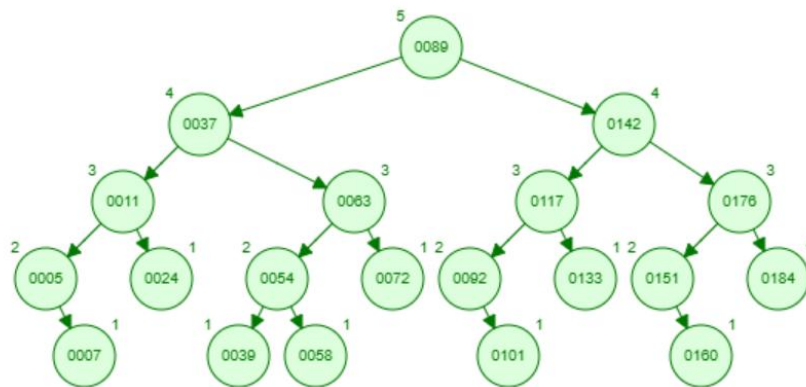
3.4 AVL Tree

Tool name / URL: <https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>

Insertion & balancing description:

在 AVL tree 中插入新節點後，會從插入的位置一路往上檢查每個父節點的高度，並計算平衡因子（左子樹高度減右子樹高度），只要某個節點的平衡因子變成 2 或 -2，就表示該節點已經失衡，需要做旋轉調整；如果失衡節點是因為「左子樹的左邊」插入而造成，就屬於 LL 情況，只要對該節點做一次右旋轉即可，如果是「左子樹的右邊」插入造成，則是 LR 情況，需要先對左子節點做左旋，再對失衡節點做右旋；相反地，若是因為「右子樹的右邊」插入而失衡，則是 RR 情況，對失衡節點做一次左旋即可，而若是「右子樹的左邊」插入造成，則是 RL 情況，需要先對右子節點做右旋，再對失衡節點做左旋，透過這些旋轉可以讓 AVL tree 恢復平衡。

Screenshot of AVL Tree (paste below):

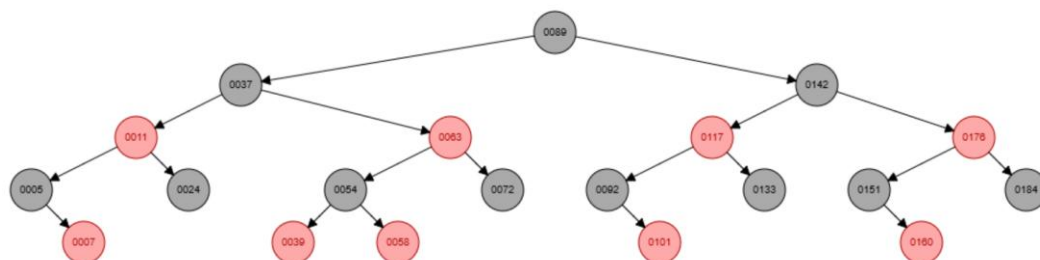


3.5 Red-Black Tree

Tool name / URL: <https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

Insertion & balancing description:

Screenshot of Red-Black Tree (paste below):



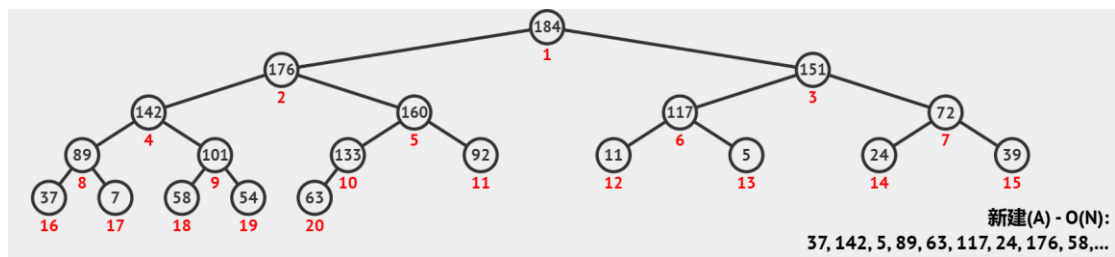
3.6 Max Heap

Tool name / URL: <https://visualgo.net/zh/heap>

Construction / heap-building description (e.g. heapify, insert-and-sift-up):

建立 max heap 時，可以用 heapify 方法把一組亂序的整數整理成堆。做法是從最後一個非葉節點 ($\text{index} = \lfloor n/2 \rfloor - 1$) 開始，對每個節點做 **sift-down**，也就是把比子節點小的節點往下移動，直到父節點 $\geq$ 任何子節點，使整棵樹符合 max heap 性質；而 **sift-up** 則是插入新節點時使用，如果新節點比父節點大，就一路往上交換，直到父節點 $\geq$ 它或到達根節點，這兩種操作都是用來維持 heap 的正確順序。

Screenshot of Max Heap (paste below):



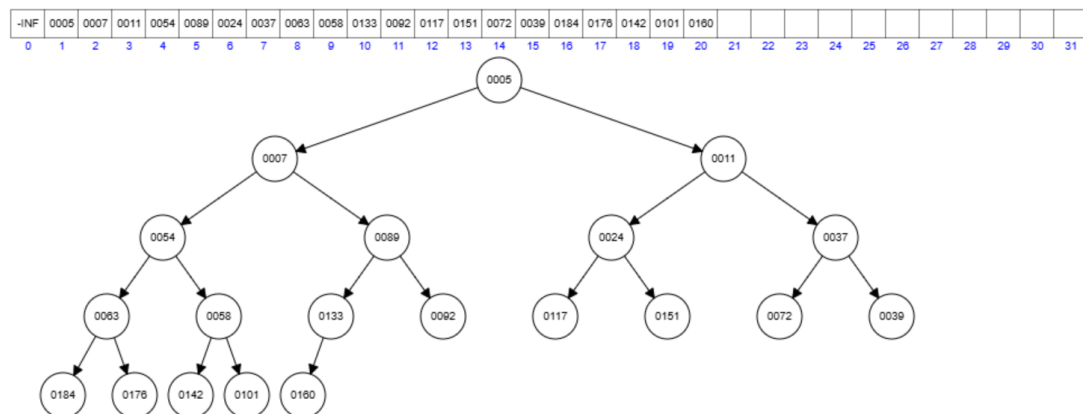
3.7 Min Heap

Tool name / URL: <https://www.cs.usfca.edu/~galles/visualization/Heap.html>

Construction / heap-building description:

建立 **min heap** 時，可以用 **heapify** 方法，把一組亂序的整數從最後一個非葉節點開始，對每個節點做 **sift-down**，也就是把比子節點大的節點往下換位置，直到它比左右子節點都小，這樣整棵樹就變成 min heap；而 **sift-up** 則是插入新節點時用的，如果新節點比父節點小，就一路往上換，直到找到合適位置或到根節點，這兩種方法都是讓 heap 保持正確順序的方法。

Screenshot of Min Heap (paste below):



Section 4. Application Examples

Tree type	Application Example (name /context)	Why this tree fits (properties that marer
Binary Tree	運算式樹，用於編譯器或計算機解析運算式	Binary Tree 能表示二元運算，內部節點對應運算子，左右子節點對應運算元，結構直觀且能透過後序走訪正確計算運算結果。

Complete Binary Tree	Heap	Complete Binary Tree 的節點由上而下、由左而右填滿，使樹可直接以陣列儲存，無需額外指標，記憶體使用效率高且能快速計算父子節點位置
Binary Search Tree	可動態插入、刪除並查詢的資料集合	BST 滿足左子樹鍵值小於根、右子樹鍵值大於根的性質，使搜尋與中序走訪能自然維持資料順序，適合需要排序與查詢的應用。
AVL Tree	查詢為主的系統	AVL Tree 嚴格限制左右子樹高度差不超過 1，確保樹高最小化，使搜尋操作能穩定維持在 $O(\log n)$ ，提供快速且可預測的查詢效能
Red-Black Tree	標準函式庫中的關聯式容器	Red-Black Tree 透過顏色規則維持近似平衡，在插入與刪除時只需少量旋轉，能在動態更新頻繁的情況下仍保證 $O(\log n)$ 的效能
Max Heap	Priority Queue	Max Heap 保證最大值位於根節點，使取得最高優先權元素的操作為 $O(1)$ ，並能在 $O(\log n)$ 時間內完成插入與刪除。
Min Heap	最短路徑演算法	Min Heap 保證最小值位於根節點，使演算法能快速選出目前距離最小的節點，並有效支援動態更新距離值。

Section 5. Reflection on Tree Family and Performance

Among BST, AVL, and Red-Black trees, which one would you pick for:

Mostly search (few updates)? Why?

AVL tree。因為 AVL tree 有最嚴格的平衡條件，確保任一節點左右子樹的高度差不超過 1，使整棵樹的高度維持在接近最小值。由於搜尋時間與樹高直接相關，AVL tree

能提供最穩定且快速的搜尋效能，其時間複雜度始終為 $O(\log n)$ 。雖然 AVL tree 在插入與刪除時可能需要較多的旋轉來重新平衡，但在更新操作很少的情況下可以忽略，因此非常適合以搜尋為主的應用場景。

Frequent insertions and deletions? Why?

Red-Black tree。因為 Red-Black tree 採用較為寬鬆的平衡條件，透過節點顏色與簡單規則來限制樹高，使其仍然維持在 $O(\log n)$ 的等級，但不追求像 AVL tree 那樣的嚴格平衡。因此在插入與刪除時，Red-Black tree 所需的旋轉與重整操作通常較少，更新成本較低，整體效能更穩定。

If you must store these 20 integers for static search only (no updates), which structure or representation would you prefer (sorted array + binary search, BST, AVL, etc.)? Why?

Section 6. AI Usage Log

Index	Date/Time	AI Service	Full Prompt/Question
1	2025/12/25	ChatGPT	請給我 definition of general tree, binary tree, complete binary tree, binary search tree, AVL tree, red-black tree, max heap and min heap 中文解釋
2	2025/12/25	Gemini	General Tree Binary Tree Complete Binary Tree BST AVL Tree Red-Black Tree Max Heap Min Heap 彼此之間的關係
3	2025/12/25	Gemini	Binary Tree 如何轉換到 BST
4	2025/12/25	Gemini	請比較 BST、AVL tree 與 red-black tree 在搜尋為主與插入刪除頻繁時的效能取捨。
5	2025/12/25	Gemini	請舉出 binary tree 的實際應用，並說明為什麼不需要排序性也能發揮作用。
6	2025/12/25	Gemini	complete binary tree 在系統或資料結構中常見於哪些地方？為什麼「完整填滿」很重要？
7	2025/12/25	Gemini	請舉一個 BST 適合的應用情境，並說明搜尋效率與結構特性之間的關係。

8	2025/12/25	Gemini	在什麼情況下會選擇 AVL tree 而不是一般 BST？請從搜尋效能角度解釋。
9	2025/12/25	Gemini	為什麼標準函式庫（如 C++ map/set）偏好使用 red-black tree？
10	2025/12/25	Gemini	請解釋為什麼 max heap 非常適合實作 priority queue。
11	2025/12/26	Gemini	請一步步說明 BST 的插入演算法，如果依照給定順序插入整數，會如何決定左右子樹？
12	2025/12/26	Gemini	請解釋 AVL tree 插入節點後，如何判斷失衡，以及 LL、LR、RL、RR rotation 各自發生的情況。
13	2025/12/26	Gemini	請用概念性方式說明 red-black tree 插入新節點時，如何透過 recoloring 與 rotation 維持性質。
14	2025/12/26	Gemini	請解釋使用 heapify 方法如何由一組整數建立 max heap，並說明 sift-up 與 sift-down 的概念。
15	2025/12/30	Chatgpt	請一步步說明 BST 的插入演算法，如果依照給定順序插入整數，會如何決定左右子樹？
16	2025/12/30	Chatgpt	請用概念性方式說明 red-black tree 插入新節點時，如何透過 recoloring 與 rotation 維持性質。
17	2025/12/30	Chatgpt	請解釋使用 heapify 方法如何由一組整數建立 min heap，並說明 sift-up 與 sift-down 的概念。
18	2025/12/30	Chatgpt	請解釋 binary tree 的插入規則，如果依照給定順序插入這 20 個整數，應該如何安排左右子樹？
19	2025/12/30	Chatgpt	red-black tree 如何在「平衡性」與「插入效率」之間取得折衷？
20	2025/12/30	Gemini	min heap 為什麼常用於排程或最短路徑相關問題？